

Portofolio
Ariko Yahya Setyawan

Klasifikasi Gambar Menggunakan Deep Learning

Convolutional Neural Network (CNN) pada Dataset CIFAR-10

Jenis Data	Gambar / Citra (Image)
Dataset	CIFAR-10 (10 kelas, 60.000 gambar 32x32 piksel)
Framework	TensorFlow / Keras
Metode	Convolutional Neural Network (CNN) dibangun dari nol

1. Tujuan

Tugas ini bertujuan untuk membangun dan melatih model deep learning yang mampu mengklasifikasikan gambar ke dalam beberapa kategori secara otomatis. Secara rinci, tujuan dari tugas ini adalah:

- Memahami tahapan lengkap pipeline klasifikasi citra menggunakan deep learning, mulai dari pengambilan data hingga evaluasi model.
- Membangun arsitektur Convolutional Neural Network (CNN) dari nol menggunakan TensorFlow/Keras.
- Melatih model pada dataset CIFAR-10 yang terdiri dari 10 kelas objek (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck).
- Mengevaluasi performa model menggunakan metrik akurasi, loss, confusion matrix, dan classification report.
- Menganalisis hasil prediksi model terhadap data uji (testing set).

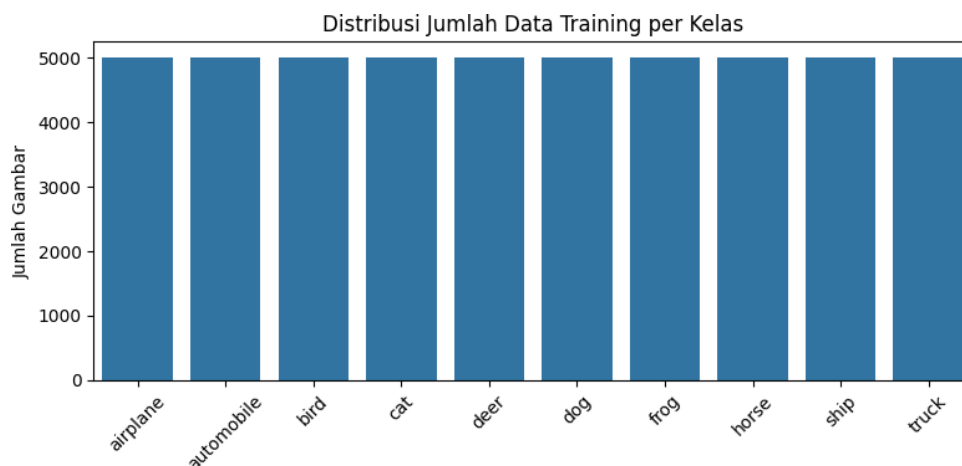
2. Dataset

Dataset yang digunakan adalah **CIFAR-10**, dataset publik populer untuk benchmark klasifikasi citra yang tersedia langsung melalui pustaka `keras.datasets`, sehingga tidak memerlukan proses unduh manual dari platform seperti Kaggle. Dataset ini terdiri dari 60.000 gambar berwarna berukuran 32x32 piksel, dengan rincian 50.000 gambar untuk data training dan 10.000 gambar untuk data testing, yang terbagi rata ke dalam 10 kelas objek.

Ukuran Data:

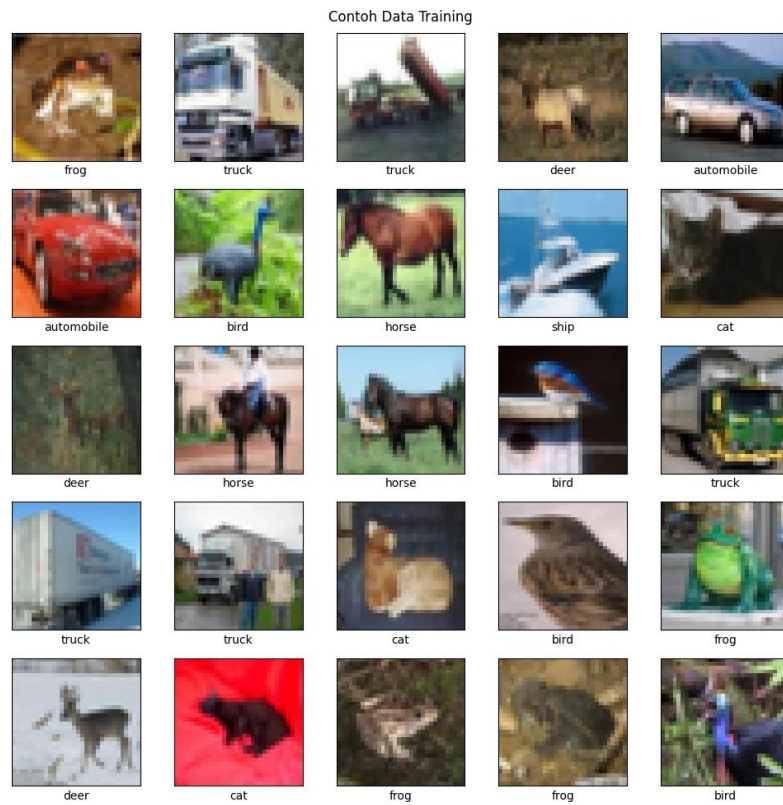
Subset	Jumlah Gambar	Dimensi
Data Training	50.000	32 x 32 x 3 (RGB)
Data Testing	10.000	32 x 32 x 3 (RGB)

Distribusi Gambar per Kelas



Gambar 1. Distribusi jumlah data training pada setiap kelas (merata, masing-masing 5.000 gambar).

Contoh Gambar dalam Dataset



Gambar 2. Contoh 25 gambar acak dari data training beserta label kelasnya.

3. Langkah-Langkah Pengerjaan

Langkah 1 — Import Library

Library utama yang digunakan meliputi TensorFlow/Keras untuk membangun dan melatih model, serta NumPy, Matplotlib, Seaborn, dan Scikit-learn untuk pengolahan data dan evaluasi.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import classification_report, confusion_matrix
```

Langkah 2 — Memuat Dataset

```
(X_train, y_train), (X_test, y_test) = keras.datasets.cifar10.load_data()

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
```

Langkah 3 — Eksplorasi Data (EDA)

Eksplorasi data dilakukan untuk melihat distribusi jumlah data per kelas serta menampilkan beberapa contoh gambar (lihat Gambar 1 dan Gambar 2 pada bagian Dataset).

Langkah 4 — Preprocessing Data

Nilai piksel dinormalisasi dari rentang [0, 255] menjadi [0, 1], dan label diubah menjadi bentuk one-hot encoding agar sesuai dengan fungsi loss `categorical_crossentropy`.

```
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

y_train_cat = keras.utils.to_categorical(y_train, num_classes=10)
y_test_cat = keras.utils.to_categorical(y_test, num_classes=10)
```

Langkah 5 — Membangun Arsitektur CNN

Model dibangun dari nol dengan 3 blok konvolusi (Conv2D + BatchNormalization + MaxPooling + Dropout) yang diikuti oleh lapisan Fully Connected untuk klasifikasi akhir ke 10 kelas.

```
model = keras.Sequential([
    layers.Input(shape=(32,32,3)),

    # Blok Konvolusi 1
    layers.Conv2D(32, (3,3), padding="same", activation="relu"),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3,3), padding="same", activation="relu"),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),

    # Blok Konvolusi 2
    layers.Conv2D(64, (3,3), padding="same", activation="relu"),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3,3), padding="same", activation="relu"),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),

    # Blok Konvolusi 3
    layers.Conv2D(128, (3,3), padding="same", activation="relu"),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.3),

    # Fully Connected
    layers.Flatten(),
```

```

layers.Dense(256, activation="relu"),
layers.BatchNormalization(),
layers.Dropout(0.5),
layers.Dense(10, activation="softmax")
])

```

Langkah 6 — Kompilasi Model dan Data Augmentation

Model dikompilasi dengan optimizer Adam dan loss categorical crossentropy. Callback EarlyStopping dan ReduceLROnPlateau digunakan agar training berhenti otomatis ketika tidak ada peningkatan, serta learning rate menurun secara adaptif. Data augmentation (rotasi, pergeseran, flip horizontal) diterapkan untuk memperkaya variasi data training.

```

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=8, restore_best_weights=True
)
reduce_lr = keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss", factor=0.5, patience=4, min_lr=1e-6
)

datagen = keras.preprocessing.image.ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
datagen.fit(X_train)

```

Langkah 7 — Training Model

Model dilatih selama maksimal 40 epoch dengan batch size 64. Proses training berhenti lebih awal pada epoch ke-29 karena EarlyStopping mendeteksi tidak ada perbaikan pada validation loss, dan bobot terbaik dikembalikan secara otomatis.

```

EPOCHS = 40
BATCH_SIZE = 64

history = model.fit(
    datagen.flow(X_train, y_train_cat, batch_size=BATCH_SIZE),
    validation_data=(X_test, y_test_cat),
    epochs=EPOCHS,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)

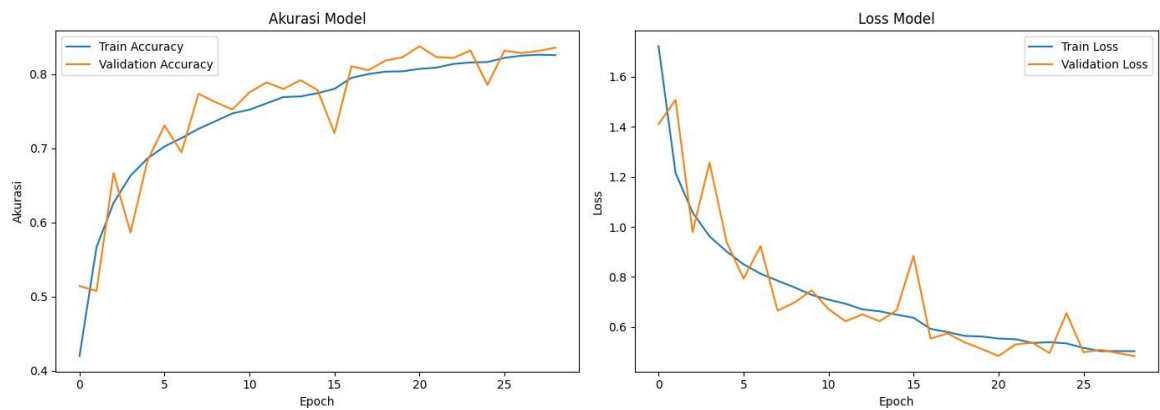
```

Epoch	Train Acc	Train Loss	Val Acc	Val Loss
1	42.00%	1.7220	51.44%	1.4111
10	74.71%	0.7280	75.23%	0.7461
17	79.50%	0.5924	81.06%	0.5538
21	80.71%	0.5535	83.75%	0.4836
29 (terbaik)	82.56%	0.5027	83.57%	0.4836

Tabel 1. Ringkasan progres training pada beberapa epoch (data lengkap tersedia pada notebook).

4. Hasil dan Evaluasi Model

Grafik Akurasi dan Loss Selama Training



Gambar 3. Kurva akurasi dan loss pada data training dan validasi di setiap epoch.

Grafik menunjukkan bahwa akurasi training dan validasi meningkat secara konsisten seiring bertambahnya epoch, sementara loss terus menurun. Tidak terdapat indikasi overfitting yang signifikan karena kurva training dan validasi bergerak beriringan.

Hasil Evaluasi pada Data Testing

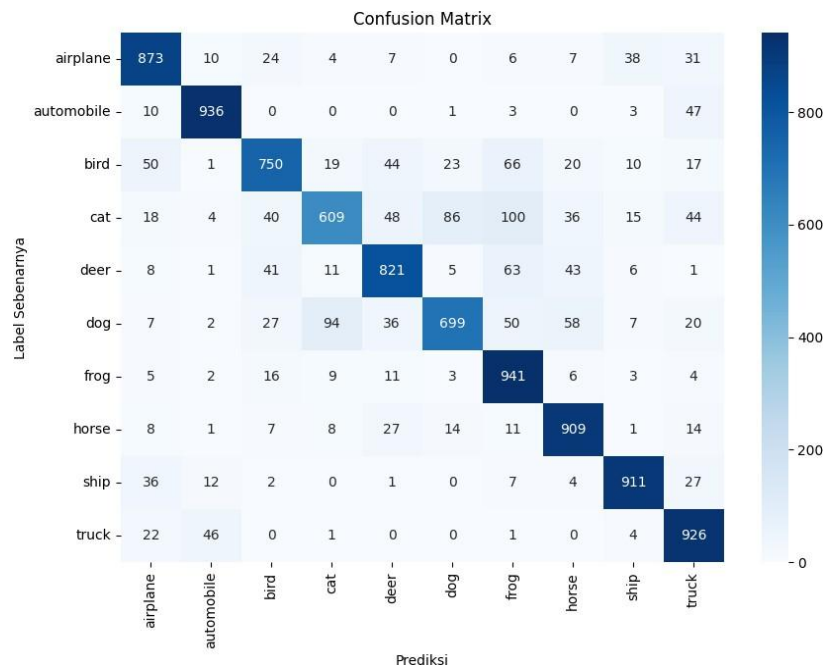
Metrik	Nilai
Akurasi (Test Accuracy)	83,75%
Loss (Test Loss)	0,4836

Classification Report

Kelas	Precision	Recall	F1-Score	Support
airplane	0.84	0.87	0.86	1000
automobile	0.92	0.94	0.93	1000
bird	0.83	0.75	0.79	1000
cat	0.81	0.61	0.69	1000
deer	0.83	0.82	0.82	1000
dog	0.84	0.70	0.76	1000
frog	0.75	0.94	0.84	1000
horse	0.84	0.91	0.87	1000
ship	0.91	0.91	0.91	1000
truck	0.82	0.93	0.87	1000
Rata-rata (weighted)	0.84	0.84	0.83	10000

Tabel 2. Precision, recall, dan F1-score untuk setiap kelas pada data testing.

Confusion Matrix

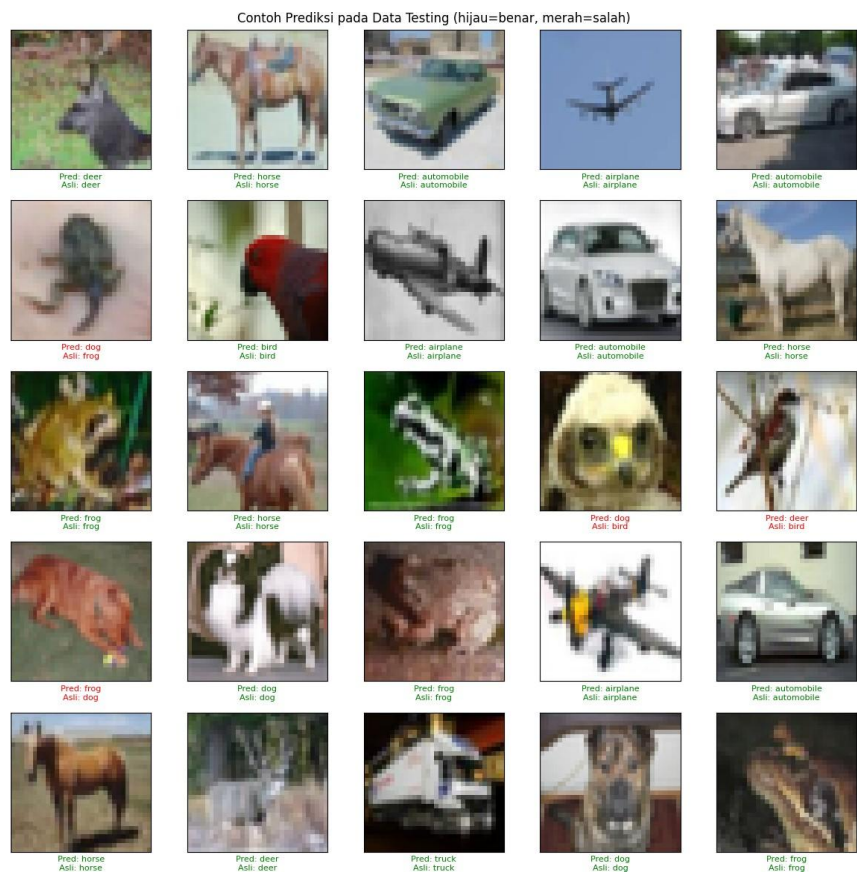


Gambar 4. Confusion matrix hasil prediksi model pada 10.000 data testing.

Dari confusion matrix terlihat bahwa kelas *cat* dan *dog* paling sering tertukar satu sama lain, yang wajar mengingat kemiripan visual antara kedua kelas tersebut. Kelas *automobile*, *ship*, dan *frog* memiliki performa klasifikasi terbaik.

5. Contoh Hasil Prediksi

Berikut adalah contoh 25 gambar acak dari data testing beserta label prediksi model dibandingkan dengan label sebenarnya. Label berwarna hijau menandakan prediksi benar, sedangkan merah menandakan prediksi salah.



Gambar 5. Contoh prediksi model pada data testing (hijau = benar, merah = salah).

6. Kesimpulan

- Model CNN yang dibangun dari nol berhasil mempelajari pola pada gambar CIFAR-10 dengan memanfaatkan beberapa blok konvolusi, batch normalization, dan dropout untuk mengurangi overfitting.
- Model mencapai akurasi sebesar **83,75%** pada data testing dengan nilai loss sebesar **0,4836**.
- Data augmentation (rotasi, pergeseran, flip horizontal) membantu meningkatkan generalisasi model terhadap data baru dan mencegah overfitting.
- Kelas yang sering tertukar (misalnya cat dan dog) terlihat pada confusion matrix, hal ini wajar karena kemiripan visual antar kelas tersebut.
- Untuk pengembangan lebih lanjut, model dapat ditingkatkan dengan menambah jumlah epoch, memperdalam arsitektur, atau menerapkan transfer learning (misalnya ResNet atau EfficientNet) untuk meningkatkan akurasi lebih jauh.